

Project Portfolio Summary

Gulsum Islamoglu

*Ira A. Fulton Schools of Engineering, Arizona State University
1151 S. Forest Ave, Tempe, AZ, USA*

gislamog@asu.edu

The following portfolio presents two projects completed during my Master of Computer Science (MCS) coursework at Arizona State University. Each project demonstrates a core competency in machine learning: unsupervised pattern discovery using clusters and supervised collision prediction in a virtual environment.

The first project, K-Means Strategy, completed in course CSE 575: Statistical Machine Learning, implements and compares the basic K-Means and enhanced K-Means++ clustering algorithms on a dataset of 300 two-dimensional points. The objective is to determine the optimal number of clusters (K) for either clustering algorithm by minimizing a loss function, which measures cluster compactness and evaluates the quality of the final centroid positions. The Elbow Method is used to determine the number of clusters that balances model complexity with diminishing improvements in clustering quality. The basic K-Means algorithm is implemented for cluster values ranging from 2 to 10. Centroids are first randomly positioned, and data points are assigned to the centroid they are closest to. At each iteration, the centroids are repositioned to the mean of their assigned points until convergence is reached, defined by no further meaningful changes in centroid positions.

To address the basic K-Means algorithm's sensitivity to random initial centroid placement, the K-Means++ algorithm is implemented. After randomly placing the initial centroid, each subsequent centroid is positioned at the data point with the maximum average distance from all existing centroids in order to maximize dispersion and cluster separation. Comparative analysis shows that the K-Means++ algorithm achieves smoother and faster convergence, with final cluster centroids more evenly distributed, especially at lower K values.

This project strengthens my skills in implementing algorithms, visualizing data, and navigating the practical trade-offs involved in unsupervised learning. It also builds my ability to connect mathematical properties of an algorithm to their practical effects. For example, the K-Means algorithm is only guaranteed to converge to a local minimum, not a global one, meaning the algorithm does not guarantee the best possible clustering, only a “good enough” solution based on the initial centroid positions. In practice, this means that a poor initial placement of centroids can lock the algorithm into a suboptimal result, which is precisely the reason the K-Means++ algorithm aims to improve the initial placement.

The second project, Neural Network for Collision Prediction, completed in course CSE 571: Artificial Intelligence, applies supervised deep learning to a robotic safety problem. Using PyTorch, sensor and action data are extracted from a virtual robot navigating its environment and processed into DataLoaders to train a fully connected neural

network to predict collisions. The data is imbalanced, with a large disparity between collision and non-collision cases; however, the full dataset is retained rather than trimmed, since this better reflects a realistic operating environment.

The architecture consists of an input layer, a 64-neuron hidden layer, a 32-neuron hidden layer, and a sigmoid output layer. Data enters through the input layer after being processed into appropriate data types, then passes through the hidden layers. The first hidden layer uses 64 neurons operating in parallel to search for simple patterns across the raw sensor and action data, expanding the range of relationships the model can detect before passing this information to the 32-neuron hidden layer, which builds on and refines these patterns to capture more complex relationships.

A ReLU activation function is applied to both hidden layers to allow the network to learn non-linear patterns. This is achieved by zeroing out negative neuron outputs, a computationally inexpensive operation that scales well to large networks. Finally, the sigmoid output layer squashes all outputs to a value between 0 and 1, representing the probability of a collision occurring.

The network is trained over 20 epochs to prevent overfitting, and the Binary Cross-Entropy Loss function is used to assess final performance. A lower loss value indicates the predicted collision probability closely matches the actual outcome during testing. After each pass through the network, the Adam optimizer updates the network weights to reduce this loss. Despite the imbalanced input data, the model achieves strong performance, with only one false positive and five missed collisions out of 1,000 test cases.

This project deepens my understanding of neural network architecture design and the forward- and back-propagation processes that adjust parameters across each epoch to improve performance. It builds my ability to select activation and loss functions that are compatible with each other and with the structure of the incoming data. It also demonstrates how machine learning can be applied in a practical, safety-critical context, such as self-driving vehicles, industrial robotics, and assistive technology, where accurately predicting hazards in real time can be critical.

Together, these two projects reflect complementary strengths in machine learning. The K-Means project demonstrates the ability to build and analyze an unsupervised algorithm, while the Neural Network project demonstrates proficiency with applying a deep learning framework to a safety-critical task. Both projects were completed independently and required end-to-end ownership of data preprocessing, model design, implementation, and analysis, reinforcing my ability to translate theoretical machine learning concepts into practical, functional systems.

K-Means Strategy Project

Gulsum Islamoglu

Ira A. Fulton Schools of Engineering, Arizona State University
1151 S. Forest Ave, Tempe, AZ, USA
gislamog@asu.edu

Abstract—Clustering is a vital technique in unsupervised machine learning, enabling the grouping of similar data points based on patterns. This project explores and compares two prominent clustering algorithms: the basic K-Means and the enhanced K-Means++. Utilizing a dataset of 300 two-dimensional points, the study aims to determine the optimal number of clusters (K) by minimizing the Loss Function, which measures the compactness of clusters. The K-Means algorithm was implemented with K values ranging from 2 to 10, revealing that an optimal K of 5 provided a balanced trade-off between cluster compactness and model complexity, as identified using the Elbow method. Next, the K-Means++ algorithm was employed to improve the initial centroid selection process, leading to more strategically dispersed centroids. Comparative analysis demonstrated that the K-Means++ method not only achieved a smoother and more consistent decrease in the Loss Function but also converged faster and produced more well-defined clusters compared to the basic K-Means method. The project was conducted individually, using algorithm development, data visualization, performance analysis, and comprehensive documentation. Through this project, skills in algorithm implementation, data analysis, and visualization were acquired, alongside a deeper understanding of clustering methodologies and their practical implications. The findings show the K-Means++ technique’s ability to enhance clustering performance, making it the preferable choice for accurate data grouping.

I. INTRODUCTION

A. Background Information

Clustering is a fundamental task in unsupervised machine learning, aimed at grouping similar data points together based on their similarities, assuming a circular group shape. Depending on the data types, the relevant similarities to base grouping on could be the Euclidean distance, Cosine similarity (relating to the angle between vectors), or shortest distance on predefined paths. When using distance to define memberships, the goal is to minimize the total distance between data points and their assigned cluster centers.

Model-based clustering has been used in the past to group geographic regions with similar rates of diet-related conditions, such as obesity and diabetes. This allows deeper exploration in the relationship between geographical areas and chronic disease prevalence, identifying regions that may need targeted interventions (Flynt & Daepf, 2015).

The K-Means algorithm is one of the most widely used clustering techniques due to its simplicity and efficiency. However, its performance and outcome depends on the initial placement of centroids, which are randomly assigned. To address this limitation, several enhancements can be made. The K-Means++ algorithm is one of those enhancements and

was developed to improve the selection of initial centroids, leading to better clustering results and faster convergence.

It's important to note that the K-Means problem is NP-hard, which means there is no known algorithm that can solve all known instances of the problem efficiently in polynomial time. This complexity makes finding the optimal solution computationally expensive for large datasets, further motivating the use of heuristic methods like K-Means++ to find near-optimal solutions with reasonable expenditures.

B. Goals and Objectives

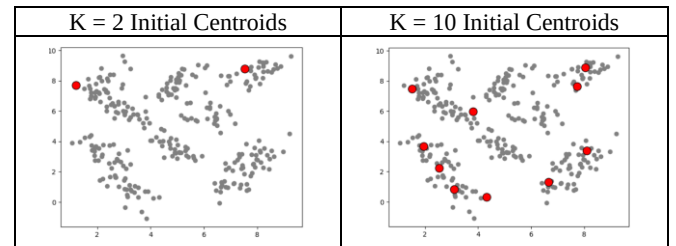
This report researches the implementation and analysis of both the basic K-Means and the enhanced K-Means++ algorithms applied to a dataset comprising 300 two-dimensional data points. The objective is to determine the optimal number of clusters (K) by minimizing the Loss Function, which quantifies the compactness of the clusters, while making sure to not overcomplicate the model or overfit the data.

II. METHODOLOGY

A. Dataset Analysis

The dataset consists of 300 data points, each represented by coordinates (x_i, y_i) . Initial visualization through scatter plots indicated the presence of approximately 5 to 7 clusters, although the exact number was to be determined mathematically using the Loss Function. For the K-Means implementation, the random initial centroid positions were given and can be visualized alongside the dataset in Table 1. For the K-Means++ implementation, the first centroid was randomly chosen, and all other centroids had to be strategically calculated.

TABLE I
Data Visualization for K-Means Algorithm



Although we did not need to preprocess the dataset in this project, it is a good idea to do so if the variables are measured in varying units or have differing magnitudes. You can normalize the data by centering the values around the mean and adjusting the range of data to ensure comparability across

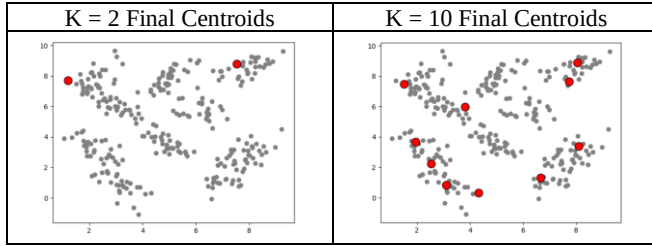
variables before applying K-Means clustering (Flynt & Dean, 2016).

B. K-Means Algorithm

The K-Means algorithm was implemented with K values ranging from 2 to 10. For each K, I assigned data points to the nearest centroid using the Euclidean distance formula. Then, I repositioned each centroid to the mean value of its assigned data points. This process was repeated until convergence was reached, which I defined as when the centroid positions changed by less than 10^{-4} .

The final centroid positions for K = 2 and K = 10 are depicted in Table 2. When K = 10, the centroids are precisely centered within their respective clusters, while for K = 2, centroids are broadly centered, capturing more generalized groupings.

TABLE 2
Data Visualization for K-Means Algorithm



C. K-Means++ Algorithm

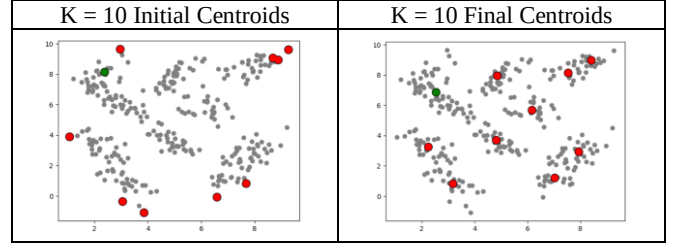
To enhance the initial centroid selection process, the K-Means++ algorithm was employed. For each value of K in the range 2 to 10, the initial centroid was randomly picked, and each subsequent centroid's position was equal to the data point with the maximum average distance from all existing centroids, so long as a centroid was not already assigned there. This would ensure that the centroids were as dispersed as possible. This approach aligns with the findings of Tanir and Nuriyeva (2017), who highlighted that the initial center selection procedure plays a crucial role in improving the clustering solution. Their method also selects the farthest points as initial centers to ensure maximum distance between them, calculated using Euclidean distance. This strategy helps to achieve better cluster separation, as demonstrated in their experiments on the Rupini dataset, where the proposed method significantly outperformed random initialization in terms of error rate.

Then, for each K value, I implemented the same iterative approach of assigning data points to their nearest centroid, then updating the centroid positions to the mean of all data points assigned to it. This process was repeated until the centroid positions stabilized (again, within a tolerance of 10^{-4}).

The graphs in Table 3 show the initial centroid positions after calculations were done to maximize their distance, as well as the final positions after convergence. The green centroid marks the initial randomly chosen centroid. Before convergence, the centroids are spread out as much as

possible, while the final centroids are centered within clusters. Upon examining the graphs, the K-Means++ algorithm appears to represent the cluster centers more accurately after convergence compared to the standard K-Means algorithm.

TABLE 3
Data Visualization for K-Means++ Algorithm



Upon examining the graphs, the K-Means++ algorithm appears to represent the cluster centers more accurately after convergence compared to the standard K-Means algorithm.

III. RESULTS AND ANALYSIS

A. Loss Function Analysis

The goal of the K-Means algorithm is to minimize the distance between each data point and its nearest centroid. After convergence, the Loss Function, defined as the sum of squared distances between data points and their assigned centroids, was used to evaluate the clustering performance for different values of K. Mathematically, it is expressed as:

$$Loss = \sum_{i=1}^k \sum_{x_i \in D_i} ||x - \mu_i||^2$$

x = data point
 μ_i = mean of cluster
 k = number of clusters

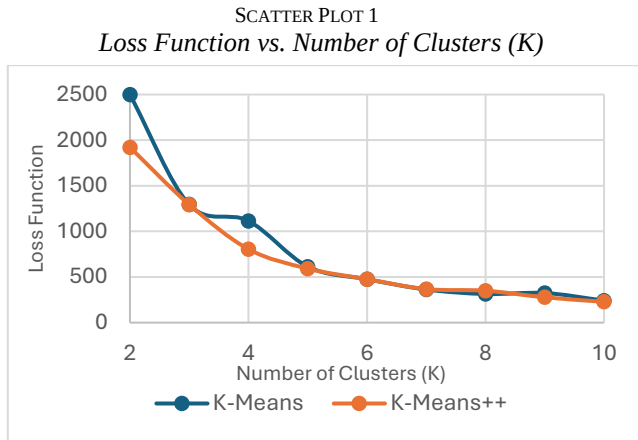
As K increases, the Loss Function consistently decreases, indicating tighter clustering. A larger value indicates the cluster is more dispersed or the centroid is not well positioned. The goal is to minimize the Loss Function as much as possible, while being considerate of overfitting and ensuring the model can generalize effectively to new, unseen data. Striking a balance between minimizing the Loss Function and maintaining model flexibility is key to achieving optimal clustering performance.

B. Comparative Analysis: K-Means vs. K-Means++

A comparative analysis between the K-Means and K-Means++ algorithms revealed that the K-Means++ method exhibited a smoother and more consistent decrease in the Loss Function as K increased. This can be visualized in Scatter Plot 1 below. The plot highlights how the K-Means++ method converges faster, leading to more stable centroid initialization and fewer iterations overall. This efficiency is particularly noticeable at lower K values.

The difference between the K-Means and K-Means++ algorithms are more pronounced at smaller K values, particularly K = 2 and K = 4. This suggests that the K-Means++ method is especially effective when fewer clusters are being formed. As K increases, the loss function consistently decreases for both algorithms, indicating that

increasing the number of clusters leads to tighter grouping of data points. However, the improvement in the loss function diminishes as K gets larger, suggesting diminishing returns.



At higher values of K ($K = 6$ and beyond), the difference between the K-Means and K-Means++ methods become smaller, suggesting that the advantages of K-Means++ diminish as more clusters are introduced. Although the loss function continues to decrease as K increases, care must be taken to avoid overfitting the data. Overly tight clustering may not generalize well to new data.

Furthermore, a decrease in the Loss Function alone is not an indicator of the quality of final centroid positions. Visually, we can see that the final centroid positions using the K-Means++ method did a better job representing the centers of clusters. The K-Means results seemed to have one centroid representing a large chunk of data points, while many centroids were cluttered together where data points were dense.

To determine the optimal number of clusters for both algorithms, the Elbow method was utilized. This method involves identifying where the rate of decrease in the Loss Function sharply slows down, forming an ‘elbow’ that indicates the optimal number of clusters. If too many centroids are chosen, the model risks overfitting—becoming too complex and capturing irrelevant noise rather than meaningful patterns. Conversely, too few centroids result in underfitting, where the model fails to capture the data’s complexity. The Elbow method involves identifying the point on the graph where the Loss Function’s rate of decrease significantly slows, indicating the most appropriate K . Scatter Plot 1 suggests that $K = 5$ is a strong candidate for the optimal number of clusters for the K-Means algorithm, as this is where the last significant decrease in the Loss Function occurs. For the K-Means++ algorithm, this can be seen as early as $K = 4$ making it the method that converges more quickly.

IV. CONTRIBUTIONS

This project was conducted as an individual effort. My contributions included developing and implementing both the basic K-Means and K-Means++ algorithms from scratch. All initial centroids for the K-Means algorithm, as well as a single

initial centroid for K-Means++ (per K value) were given. Graphs and scatter plots to visualize the data distribution and Loss Function trends were developed by me. Further, I conducted comparative analysis between the two algorithms, outlining their effectiveness and drawing inferences from raw data.

V. SKILLS AND KNOWLEDGE ACQUIRED

Throughout the completion of this project, I acquired and strengthened several skills and techniques. I gained the ability to create a machine learning algorithm to effectively group two-dimensional data points based on their distance from each other. Then, I enhanced my ability to utilize data visualization tools to draw inferences from data. The project allowed me to gain a deeper understanding of the implications of algorithmic choices, such as centroid initialization, on clustering performance. Finally, it strengthened my problem-solving skills by allowing me to create my own definition of convergence and understand the meaning behind the results I found. Through this project, I found that AI became much less convoluted for me as I developed a machine learning model from scratch and assessed its capabilities.

VI. CONCLUSION

This project successfully demonstrated the implementation and a comparative analysis of the basic K-Means and the enhanced K-Means++ algorithms on a dataset consisting of 300 two-dimensional points. The K-Means++ algorithm provided superior clustering performance by implementing a more strategic initial placement of centroids, leading to faster convergence and more compact and well-defined clusters. The Elbow method identified $K = 5$ as the optimal number of clusters for the K-Means algorithm, and $K = 4$ for the K-Means++ algorithm.

Although the K-Means++ algorithm optimized our results, it may be more sensitive to outliers. A centroid would likely be assigned to the position of an outlier due to its distance from the rest of the data points, which would skew the results unpredictably. This could potentially be prevented by preprocessing the data by removing or transforming outliers prior to applying the algorithm.

The K-Means algorithm is guaranteed to converge to a local minimum, but not necessarily a global minimum. This means that the Loss Function is not guaranteed to be the absolute minimum across the entire data space, but rather only with neighboring points. The AS 113 algorithm was developed to enhance clustering performance by swapping cluster assignments of pairs of data points to see if it improves the Loss Function. This algorithm aims to potentially find a global minimum, albeit at a high computational cost (Hartigan & Wong, 1979).

Other algorithms have also been developed to enhance the basic K-Means method. The Agglomerative method starts by assigning each data point as its own cluster, then iteratively merges the closest clusters until only the desired number of clusters remain. Oppositely, the Divisive method initially

assigns all data points to one cluster, and iteratively splits the cluster until the desired number of clusters is reached.

REFERENCES

- [1] A. Flynt and M. I. Daepp, "DIET-related chronic disease in the Northeastern United States: A model-based clustering approach," *International Journal of Health Geographics*, vol. 14, no. 1, Sep. 2015. doi:10.1186/s12942-015-0017-5
- [2] A. Flynt and N. Dean, "A Survey of Popular R Packages for Cluster Analysis," *Journal of Educational and Behavioral Statistics*, vol. 41, no. 2, pp. 206–208, Apr. 2016. doi:10.3102/1076998616631743
- [3] D. Tanir and F. Nuriyeva, "On selecting the initial cluster centers in the K-means algorithm," *2017 IEEE 11th International Conference on Application of Information and Communication Technologies (AICT)*, vol. 2, pp. 1–5, Sep. 2017. doi:10.1109/icaict.2017.8687081
- [4] J. A. Hartigan and M. A. Wong, "Algorithm as 136: A K-Means Clustering Algorithm," *Applied Statistics*, vol. 28, no. 1, p. 102, 1979. doi:10.2307/2346830

Neural Network for Collision Prediction Project

Gulsum Islamoglu

*Ira A. Fulton Schools of Engineering, Arizona State University
1151 S. Forest Ave, Tempe, AZ, USA
gislamog@asu.edu*

Abstract—This project explores the application of neural networks in robotics, focusing on collision prediction to enable safe navigation in simulated environments. Initially, training data is collected using a simulated robot in a virtual environment. Data is collected from its sensor readings, the action performed, then whether or not a collision occurred. The data is then processed into DataLoaders using PyTorch, an open-source machine learning framework, to maintain balanced data distribution and to convert the data into a format compatible with PyTorch modules. Next, the data is processed through a custom neural network architecture. Finally, the model is trained to predict and prevent collisions with high accuracy. The project allows for hands-on learning of data collection, preprocessing, neural network implementation, and performance evaluation of the model. The final model demonstrated its ability to minimize collisions effectively, contributing to machine learning applications in robotics safety. Through this project, I gained knowledge in how data is processed through the layers of a neural network, as well as how to apply appropriate activation and loss functions to successfully train the model and evaluate its performance.

I. INTRODUCTION

A. Background Information

Artificial intelligence, machine learning, and robotics are constantly converging to solve complex tasks such as autonomous navigation in dynamic environments. As the fields advance, collision avoidance in unknown environments have become essential features for applications such as self-driving cars, industrial robots, and assistive devices. These systems require accurate models to process sensor data and make real-time decisions to prevent accidents. Because of its ability to learn complex patterns from data, neural networks have become a cornerstone in completing this task.

PyTorch can help facilitate the design and training of a neural network capable of handling this problem. PyTorch is a framework developed by Meta AI and is widely used for deep learning and machine learning tasks. It has prebuilt modules to create and pass through custom neural network layers, handle large datasets, and apply optimizers.

B. Goals and Objectives

This project integrates AI concepts and robotics simulation to train a model capable of predicting collisions based on sensor readings and actions, representing a practical application of machine learning in a robotics context.

This project leverages PyTorch to build and train a collision prediction model for a simulated robot. After raw data is extracted from simulations, it is turned into DataLoaders, an object representing your data that can be utilized by PyTorch. Then, the data is processed, a model

architecture is designed, and passes through the network are defined.

The model was then trained using the binary cross-entropy loss function and optimized with the Adam (Adaptive Moment Estimation) optimizer, both provided by PyTorch. It was then evaluated using testing datasets to ensure robustness. Finally, the model's effectiveness was validated by enabling the robot to navigate the environment to reach an endpoint and documenting collisions, with the aim of having as few collisions as possible. This approach provides an opportunity to combine theoretical knowledge with practical implementation to address challenges in robotics using machine learning.

II. METHODOLOGY

A. Part 1: Dataset Analysis

In this section, the dataset was collected using a simulated robot equipped with five distance sensors placed at fixed angular positions relative to its body: 66° , 33° , 0° , -33° , and -66° . These sensors measure distances to obstacles within a 150-pixel range.

The dataset consists of seven columns: five for the sensor readings, one for the action performed by the robot, and one for whether a collision occurred (0 for no collision, and 1 for collision). Figure 1 displays the simulation in action, collecting data as the robot navigates its environment.

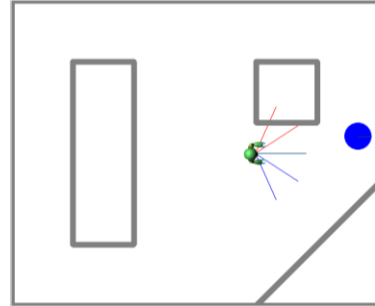


Fig. 1. Robot Navigation Simulation

The final dataset is a CSV file containing 11,000 samples, with 9206 representing non-collision cases (0) and 1794 representing collision cases (1), resulting in an approximate 5:1 imbalance ratio.

B. Part 2: Data Preprocessing

Although there is a large 5:1 imbalance ratio, this dataset accurately reflects the real environment of the robot, where collisions are less frequent than non-collisions. Despite this imbalance, the dataset provides enough collision samples to train the model, and I found it was not necessary to balance their distribution.

PyTorch DataLoaders were then used to manage and preprocess the dataset efficiently. The dataset was split into training and testing subsets to evaluate the model's performance. The DataLoader iteratively fed data to the model in batches consisting of 16 samples each, which helped with memory usage by processing smaller chunks of data at a time.

This batch-wise data processing \ enables the model to calculate gradients and update its parameters incrementally during backpropagation. The gradients tell the model how much each parameter contributed to the error and allow the model to update them to perform better.

C. Part 3: Neural Network Architecture

Neural networks are composed of multiple layers that perform simple transformations, where the output of one layer provides the input for the next. They begin at an generalized level and continue to get more refined. Increasing the number of layers enables the model to recognize more intricate patterns to complete more complex tasks [3].

Our neural network was created using PyTorch's torch.nn module. It is designed to predict the likelihood of a collision based on the robot's sensor readings and actions. It consists of an input layer, two hidden layers, and an output layer.

The input layer consists of six inputs – five for the sensor readings from the simulated robot, and one for the action representing the robot's steering adjustment. This input data captures the robot's current environment and status, providing the model with enough information to predict potential collisions.

Next, the two fully connected hidden layers are designed to extract patterns and relationships from the input data. The first hidden layer contains 64 neurons and the second contains 32. Neurons are the building blocks of a neural network - each takes an input, multiplies it by weights, adds a bias, then passes the result through the ReLU (Rectified Linear Unit) activation function. The ReLU activation function keeps the positive input and zeroes out the negative. This simple comparison to zero makes it computationally simple and efficient.

Finally, the output layer has a single neuron to predict the possibility of a collision. The Sigmoid activation function is used to shape the output into a probability showing the likelihood of a collision occurring.

The learning rate used was 0.001, meaning that each parameter would update by 0.1% of the calculated gradient's value. The gradients indicate the direction and magnitude needed to minimize the model's error.

D. Part 4: Training and Optimization

Training was conducted over 20 epochs, with loss values monitored for both training and testing datasets. The model's performance improved significantly during the initial epochs and stabilized as the number of epochs increased. Limiting training to 20 epochs ensured computational efficiency while achieving strong performance. Extending training beyond 20 epochs could risk overfitting, where the model starts to

memorize the training data instead of retaining the capability of learning generalizable patterns for unseen data.

The Binary Cross-Entropy Loss (BCELoss) function was used to evaluate the performance of the neural network. This loss function is compatible with the Sigmoid activation function because it expects the output to be a probability, which the Sigmoid function provides. BCELoss measures the difference between the predicted probability and the actual collision status, penalizing predictions more heavily if the model is confident but wrong, such as if it shows a high probability of a collision occurring but no collision occurs.

This loss function helps the model learn from mistakes by adjusting weights for incorrectly predicted samples. Further, its use of the log function ensures smooth gradients for stable training. The function, as seen in (1), calculates a high loss if the collision probability is far from the actual collision result, and a low loss if it is close. Here, y_i is the actual collision result (0 or 1), $\hat{y}_i$ is the predicted probability (or output of sigmoid function, between 0 or 1), and N is the total number of samples.

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (1)$$

During each batch during the forward pass through the network, the loss function is calculated. During the backward pass, or backpropagation process, the Adam optimizer is used to calculate gradients to reduce this loss. More specifically, the optimizer figures out how much each weight and bias contributed to the error and updates the gradients, which tells you how to adjust the weights for more accurate performance.

The Adam optimizer smooths out noisy gradients to avoid large jumps in updates and to make the optimization process smoother and more stable. It also adjusts the learning rates for each parameter, so important ones get bigger updates, and insignificant ones get smaller updates. These techniques allow the Adam optimizer to learn faster because it does not rely on a fixed value. Although the regular Adam optimizer is sufficient for this non-critical task, there exists an optimized version called ND-Adam (Normalized Direction-Preserving Adam), which preserves the direction of gradient and normalized the weight magnitudes. Rather than updating weights individually, it groups them based on related parameters and updates them together. This would ensure the updates follow the same overall gradient direction, improving the optimization process [4].

III. RESULTS AND ANALYSIS

A. Part 1: Data Insights

The number of samples in the dataset, 11,000, proved to be sufficient to train the learning model. This high number of samples was likely the reason the model successfully wandered the virtual environment with high accuracy despite a 5:1 imbalance ratio between non-collision and collision cases.

B. Part 2: Preprocessing Results

The use of DataLoaders enabled sufficient data handling and batch-wise processing. The batch size of 16 proved to be sufficient for our case and potentially lead to better

generalization in the model, making it more likely to perform well on unseen data. A larger batch size could have led the model to see too many examples at once, leading it to average out the data and find a solution that works perfect for training data but struggles with new data.

C. Part 3: Network Performance

The chosen architecture effectively captured patterns in the sensor readings and actions, enabling accurate collision predictions. The two hidden layers served to refine the model. The first layer, with 64 neurons, focused on learning simple patterns or relationships in the data, such as direct correlations. It had a higher number of neurons to ensure it captured a wide range of features and relationships. The second layer built upon those patterns to extract more complex features. Having only two hidden layers strikes a balance between having a simple and easy to train model and sufficient knowledge to complete the task.

Because the ReLU function does not compress its output into small ranges like the Sigmoid and Tanh functions, its gradients remain larger and more consistent, effectively avoiding the vanishing gradient problem. However, ReLU is not without its challenges. One significant problem is the dying neuron issue, where neurons become inactive and output zero for any input, rendering them useless. For example, if the weights are initialized with values that cause many inputs to fall below zero, the ReLU function will output zero for those neurons. Once a neuron outputs zero, it stops contributing to learning because its gradient becomes zero during backpropagation [2]. Despite this, ReLU's ability to introduce non-linearity allowed the model to learn complex patterns effectively.

The learning rate of 0.1% proved effective. A learning rate that is too high (10%) may cause the model to overshoot the optimal solution, and a rate that is too low (0.001%) will make training extremely slow. The chosen rate ensured steady progress towards reducing loss.

D. Part 4: Loss and Accuracy

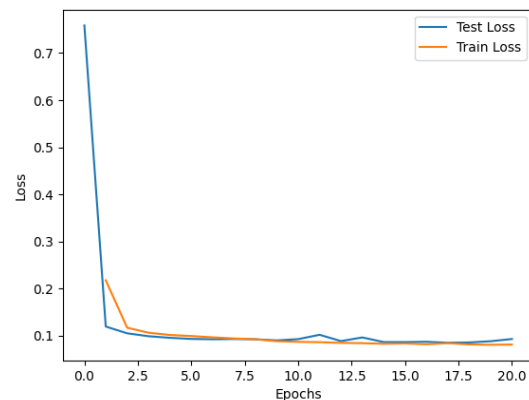
The loss function trend provides insights into the model's learning progress, convergence behavior, and overall effectiveness. A higher loss value depicts deviation between the predicted collision probability and the actual collision status. The model started with an initial Test Loss of 0.7586, indicating significant room for improvement. This loss value reflects the starting state of the model, where it has not yet learned meaningful patterns from the dataset.

By the end of the first epoch, both the training and testing losses decreased dramatically, with a Train Loss of 0.2178 and Test Loss of 0.1196. This sharp and sudden decline demonstrates the model's ability to quickly adapt and learn meaningful relationships from the input data. It highlights the effectiveness of the Adam optimizer and the network's architecture in quickly learning key features.

From epoch 2 onward, both losses continued to steadily but minimally decrease, reaching a final Train Loss of 0.0814 and Test Loss of 0.0932. The training loss converged smoothly, reflecting consistent updates to the model's parameters and gradients. The testing loss, however, had minor variations and jumps after epoch 10, suggesting some

sensitivity to the testing data, though the model maintained overall stability. Graph 1 shows a visual depiction of the loss function values for each epoch for both training and loss data.

GRAPH 1
LOSS VS. EPOCHS FOR TRAINING AND TESTING DATA



The final evaluation metrics indicated high accuracy, with only 1 false positive and 5 collisions out of 1000, demonstrating the model's success in predicting collision risk. This could further be refined by collecting more data from the simulation or adding more layers into the network.

IV. CONTRIBUTIONS

This project was completed individually, with my contributions including data collection and processing, as well as the development, implementation, and refinement of the neural network architecture. I designed and trained the model using the Binary Cross-Entropy Loss function and the Adam optimizer, ensuring convergence and stability by the end of the 20 epochs. Further, I generated visualizations, including a graph to depict training and testing loss trends, and conducted an analysis of the model's performance to assess its effectiveness in predicting collision probabilities. However, I did not create the robotic simulation, which includes the steering behaviors of the robot, sensors and collision detection, or the simulated environment.

V. SKILLS AND KNOWLEDGE ACQUIRED

Throughout this project, I gained valuable skills and knowledge in several areas of machine learning and software development. I learned how to implement a neural network architecture using Meta AI's PyTorch library in the python language, including input, hidden, and output layers. Furthermore, I applied appropriate activation functions based on the requirements of the task or layer. I acquired hands-on experience with the Adam optimizer, understanding its role in weight and bias updates, and how adaptive rates can improve training efficiency.

I've gained a deeper understanding of the Binary Cross-Entropy Loss function and its role in binary classification tasks. I've personally graphed the loss function to better understand its behavior. Lastly, I developed skills in analyzing the loss trends and identifying signs of convergence. This project provided a comprehensive learning experience, solidifying my knowledge of theoretical concepts through practical implementation. It has enhanced my ability to solve real-world problems using machine learning.

VI. CONCLUSION

This project successfully demonstrated the benefits of applying neural networks to address collision prediction in a simulated robotic environment. By leveraging PyTorch for data processing, model implementation, and optimization, the project highlighted how machine learning can be effectively applied to real-world problems. The model's ability to rapidly reduce loss and stabilize performance over 20 epochs reflects the robustness of the chosen architecture, loss function, and optimizer.

While the neural network developed in this project effectively predicts collisions based on sensor readings and steering actions, further advancements could integrate techniques such as fuzzy logic to enhance the model's navigation capabilities. Fuzzy logic excels in handling uncertainty and imprecise inputs, such as real-time sensor data in dynamic environments. By combining the pattern recognition strengths of neural networks with the adaptability of fuzzy logic, future models could achieve more context-aware decision-making capabilities [1]. This hybrid approach would expand the model's utility in diverse and unpredictable real-world applications.

The insights and methodologies developed in this project have practical implications for future advancements in robotics and autonomous systems. Collision prediction is a critical component in technologies that are quickly advancing in our world today. For example, it can enable self-driving cars to navigate complex environments more safely and make split second decisions to save lives in the case of accidents or unsafe road conditions. In industrial environments, it can prevent robots from damaging equipment and improve workplace safety. In healthcare, it can deliver patient meals or medications safely and respond to medical emergencies.

Beyond robotics, the neural network design and training process explored here can be expanded to contribute to other fields, such as drone navigation. This project not only bridges theoretical knowledge and practical implementation, but also sets a foundation for broader applications of machine learning.

REFERENCES

- [1] A. Pandey, "Mobile Robot Navigation and Obstacle Avoidance Techniques: A Review," *International Robotics & Automation Journal*, vol. 2, no. 3, May 2017. doi:10.15406/iratj.2017.02.00023
- [2] L. Lu, Y. Shin, Y. Su, and G. E. Karniadakis, "Dying ReLU and Initialization: Theory and Numerical Examples," *arXiv preprint*, arXiv:1903.06733, 2019. [Online]. Available: <https://arxiv.org/pdf/1903.06733>
- [3] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, pp. 436–444, May 2015. doi: 10.1038/nature14539. [Online]. Available: <https://www.nature.com/articles/nature14539>
- [4] Z. Zhang, "Improved Adam Optimizer for Deep Neural Networks," in *2018 IEEE/ACM 26th International Symposium on Quality of Service (IWQoS)*, Banff, AB, Canada, 2018, pp. 1-2. doi: 10.1109/IWQoS.2018.8624183.